Name - Raghav Yadav
class - Krg - 2B
UID - 23BCS11935

Surprise Test

__/__/____

Competitive Coding

Ques. Given three integers n, a and b return n^{th} magical number. Ans may be very large return $10^9 + 7$ %.
Magical no. - If no. either divisible by a or b
n = 1 , a = 2 , b = 3
Output = 2.

Brute Force

Algo:-
- Take n, a and b as input
- Run a loop while its true
- Make a counter and when no. either divisible by a or b increment counter
- And when counter = = n then return i

B

Code

```
# include <iostream>
using namespace std;

int main() {
  int n;
long long a, b, 1 i, counter;
   cin >> a>> b;
i = 1; counter = 0;
while(true) {
 if( i % a == 0) {

cout << n <<"th term is:" << i << endl;
return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
 int n, a, b
long long i, counter;
 cin >> n >> a>> b;
 i = 1
 counter = 0;
 while (true) {
  if (counter == n){
       break;
  }
  if( i % a ==0) {
     counter ++; }
  else if( i % b == 0 && i % a != 0)
     counter ++;
 }
 i ++;
}
```

- Time complexity → O(i)
- Space complexity → O(1)

Optimal ~~Sol~~ Soln

Algo- Take Input n, a, b.
→ Calculate min(n*a, n*b)
→ Set high = min(n*a, n*b).
→ Set low = 0, count = 0
→ Run a loop
→ Inside loop find mid of high
→ check if (LCM(mid) >, count) then ~~mid~~ high = mid
→ if (LCM(mid) < count) then low = mid.
→ Repeat above 2 process until ~~count~~ LCM(mid) = count = n;

Code

```
#include<iostream>
using namespace std;
long long gcd (long long a, long long b) {
    while (b) {
        long long t = a % b;
        a = b;
        b = t;
    } return a;
}
int nthNum( int n, int a, int b){
    const int MOD= 1000000007;
    long long l = min(a, b), r =(long long)n * min(a, b);
    long long lcm = (long long) a* b / gcd (a, b);
    while ( l<r){
        long long m = l + (r-l)/2;
        if ( m/a + m/b - m/lcm <n)
            l =m+1;
        else
            r= m;
    }
    return l% MOD;
}
```